Available online at www.sciencedirect.com
ScienceDirect
Pattern Recognition 40 (2007) 964–971
PATTERN RECOGNITION
THE JOURNAL OF THE PATTERN RECOGNITION SOCIETY
www.elsevier.com/locate/pr

An improved incremental training algorithm for support vector machines using active query

Shouxian Cheng, Frank Y. Shih*

Computer Vision Laboratory, College of Computing Sciences, New Jersey Institute of Technology, Newark, NJ 07102, USA

Received 13 December 2005; received in revised form 29 May 2006; accepted 8 June 2006

Abstract

In this paper, we present an improved incremental training algorithm for support vector machines (SVMs). Instead of selecting training samples randomly, we divide them into groups and apply the *k*-means clustering algorithm to collect the initial set of training samples. In active query, we assign a weight to each sample according to its confidence factor and its distance to the separating hyperplane. The confidence factor is calculated from the error upper bound of the SVM to indicate the closeness of the current hyperplane to the optimal hyperplane. A criterion is developed to eliminate non-informative training samples incrementally. Experimental results show our algorithm works successfully on artificial and real data, and is superior to the existing methods.

Keywords: Incremental training; Active learning; Support vector machine; Clustering algorithm; Pattern classification

1. Introduction

The *support vector machine* (SVM) is intended to generate an optimal separating hyperplane (OSH) by minimizing the generalization error without using the assumption of class probabilities such as Bayesian classifier [1–4]. The decision hyperplane of SVM is determined by the most informative data instances, called *support vectors* (SVs). In practice, these SVs are a subset of the entire training data. By applying feature reduction in both input and feature space, a fast non-linear SVM can be designed without a noticeable loss in performance [5,6].

The training procedure of SVMs usually requires huge memory space and significant computation time due to the enormous amounts of training data and the quadratic programming problem. Some researchers proposed incremental training or active learning to shorten the training time [7–12]. The goal is to select a subset of training samples while preserving the performance as using all the training samples. Instead of learning from randomly selected samples, the active learning queries the informative samples in each incremental step.

Syed et al. [8] developed an incremental learning procedure by partitioning the training dataset into subsets. At each incremental step, only SVs are added to the training set at the next step. Campbell et al. [9] proposed a query learning strategy by iteratively requesting the label of a sample which is the closest to the current hyperplane. Schohn and Cohn [7] presented a greedy optimal strategy by calculating class probability and expected error. A simple heuristic is used for selecting training samples according to their proximity with respect to the dividing hyperplane.

Moreover, An et al. [10] developed an incremental learning algorithm by extracting initial training samples as the margin vectors of two classes, selecting new samples according to their distances to the current hyperplane, and discarding the training samples that do not contribute to the separating plane. Nguyen and Smeulders [11] pointed out that prior data distribution is useful for active learning, and the new training data selected from the samples have the maximal contribution to the current expected error.

* Corresponding author.
E-mail address: shih@njit.edu (F.Y. Shih).

0031-3203/$30.00
doi:10.1016/j.patcog.2006.06.016

Most existing methods of active learning perform the measurement of proximity to the separating hyperplane. Mitra et al. [12] presented probabilistic active learning using a distribution determined by the current separating hyperplane and confidence factor. Unfortunately, they did not provide the criterion for choosing the test samples. There are two issues in their algorithm. First, the initial training samples are selected randomly which may generate the initial hyperplane to be far away from the optimal solution. Second, using only the SVs from the previous training set may lead to bad performance.

In this paper, we present an improved incremental training algorithm for SVMs using active query. In active query, samples are selected according to their weights calculated from the confidence factor and their distances to the separating hyperplane. We define the confidence factor as the closeness of the current hyperplane to the optimal hyperplane. A new method to calculating the confidence factor from the error upper bound of the SVM is also presented. The improvements reside in collecting by k-means algorithm the appropriate set of training samples and in the adoption of a new criterion to eliminate the non-informative training samples incrementally. The rest of the paper is organized as follows. In Section 2, we present the improved incremental training algorithm. In Section 3, we provide experimental results and comparisons with other active learning methods. Finally, conclusions are given.

2. The improved incremental training algorithm

In this section, the introduction of SVM, the selection of initial training samples using the k-means clustering algorithm, the active query of most informative samples, and the criterion of eliminating non-informative training samples are presented.

2.1. Introduction of SVM

Suppose we have l patterns, and each pattern consists of a pair: a vector $\mathbf{x}_i \in \mathbf{R}^n$ and the associated label $y_i \in \{-1, 1\}$. Let $X \subset R^n$ be the space of patterns, $Y = \{-1, 1\}$ be the space of labels, and $p(\mathbf{x}, y)$ be a probability density function on $X \times Y$. The machine learning problem consists of finding a mapping: $\text{sign}(f(\mathbf{x}_i)) \to y_i$, which minimizes the error of misclassification. Let f denote the decision function. The sign function is defined as

$$\text{sign}(f(\mathbf{x})) = \begin{cases} 1, & f(\mathbf{x}) \geqslant 0, \\ -1 & \text{otherwise.} \end{cases} \tag{1}$$

The expected value of the actual error is given by

$$E[e] = \int \frac{1}{2}|y - \text{sign}(f(\mathbf{x}))|\, p(\mathbf{x}, y)\, \mathrm{d}\mathbf{x}. \tag{2}$$

However, $p(\mathbf{x}, y)$ is unknown in practice. We use the training data to derive the function f whose performance is controlled by two factors: the training error and the capacity measured by Vapnik Chervonenkis (VC) dimension [3]. Let $0 \leqslant \eta \leqslant 1$. The following bound holds with a probability of $1 - \eta$:

$$e \leqslant e_t + \sqrt{\left(\frac{h(\log(2l/h) + 1) - \log(\eta/4)}{l}\right)}, \tag{3}$$

where h is the VC dimension and e_t is the error tested on the training set

$$e_t = \frac{1}{2l}\sum_{i=1}^{l} |y_i - \text{sign}(f(\mathbf{x}_i))|. \tag{4}$$

For two-class classification, SVMs use a hyperplane that maximizes the margin (i.e., the distance between the hyperplane and the nearest sample of each class). This hyperplane is viewed as the OSH. The training patterns are linearly separable if there exists a vector $\mathbf{w}$ and a scalar b such that the inequality

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geqslant 1, \quad i = 1, 2, \ldots, l \tag{5}$$

is valid. The margin between the two classes can be simply calculated as

$$m = \frac{2}{|\mathbf{w}|}. \tag{6}$$

Therefore, the OSH can be obtained by minimizing $|\mathbf{w}|^2$ subject to the constraint in Eq. (5).

The positive Lagrange multipliers $\alpha_i \geqslant 0$ are introduced to form the Lagrangian as

$$L = \frac{1}{2}|\mathbf{w}|^2 - \sum_{i=1}^{l} \alpha_i y_i(\mathbf{x}_i \cdot \mathbf{w} + b) + \sum_{i=1}^{l} \alpha_i. \tag{7}$$

The optimal solution is determined by the saddle point of this Lagrangian, where the minimum is taken with respect to $\mathbf{w}$ and b, and the maximum is taken with respect to the Lagrange multipliers α_i. By taking the derivatives of L with respect to $\mathbf{w}$ and b, we obtain

$$\mathbf{w} = \sum_{i=1}^{l} \alpha_i y_i \mathbf{x}_i, \tag{8}$$

$$\sum_{i=1}^{l} \alpha_i y_i = 0. \tag{9}$$

Substituting Eqs. (8) and (9) into Eq. (7), we obtain

$$L = \sum_{i=1}^{l} \alpha_i - \frac{1}{2}\sum_{i=1}^{l}\sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j. \tag{10}$$

Thus, constructing the optimal hyperplane for the linear SVM is a quadratic programming problem which maximizes Eq. (10) subject to Eq. (9).

If the data are not linearly separable in the input space, a non-linear transformation function $\mathbf{\Phi}(\cdot)$ is used to project $\mathbf{x}_i$ from the input space $\mathbf{R}^n$ to a higher dimensional feature space $\mathbf{F}$. The OSH is constructed in the feature space by maximizing the margin between the closest points $\mathbf{\Phi}(\mathbf{x}_i)$. The inner-product between two projections is defined by a kernel function $K(\mathbf{x}, \mathbf{y}) = \mathbf{\Phi}(\mathbf{x}) \cdot \mathbf{\Phi}(\mathbf{y})$. The commonly used kernels include polynomial kernel, Gaussian *radial basis function* (RBF) kernel, and Sigmoid kernel. Similar to linear SVM, constructing the OSH for SVM with kernel $K(\mathbf{x}, \mathbf{y})$ involves maximizing

$$L = \sum_{i=1}^{l} \alpha_i - \frac{1}{2} \sum_{i=1}^{l} \sum_{j=1}^{l} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j), \tag{11}$$

subject to Eq. (9).

In practical applications, the training data in two classes are not often completely separable. In this case, a hyperplane that maximizes the margin while minimizes the number of errors is desirable. It is obtained by maximizing Eq. (11) subject to the constraints $\sum_{i=1}^{l} \alpha_i y_i = 0$ and $0 \leqslant \alpha_i \leqslant C$, where α_i is the Lagrange multiplier for pattern $\mathbf{x}_i$ and C is the regularization constant that manages the tradeoff between the minimization of the number of errors and the necessity to control the capacity of the classifier.

2.2. Selection of initial training samples

In most of the incremental training methods for SVMs, the initial training samples are selected randomly [7,9,12]. In a simple case of positive and negative training samples distributed over two separable clusters, it is suitable to use a random selection. However, in the complex case such as the samples of a class distributed over several clusters, it is likely to generate the final hyperplane which is far away from the optimal solution.

Fig. 1 shows a case of artificial data containing two classes: one with 1000 points labeled as "+" and the other with 1100 points labeled as "o." The "o" points are distributed over two clusters: one with 1000 points and the other with 100 points. The dash line indicates the decision boundary of the SVM training using 1000 "o" and 1000 "+" training samples. The solid line indicates the decision boundary of using 1100 "o" and 1000 "+" training samples. It is observed that the solid line represents the correct hyperplane.

In the incremental training, we query the samples near the hyperplane of the current SVM and add them to the current training set to form the new training set for the next step. If we select the initial training samples randomly, we may obtain a poor decision boundary as the dash line in Fig. 1. We have conducted this experiment 10 times by taking a random 10% of the samples in each class as the initial training samples. Results show the dash line is chosen four times and the solid line is chosen six times.

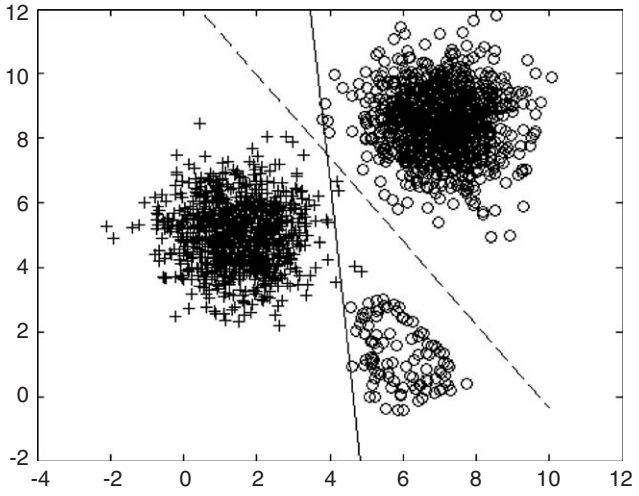

Fig. 1. A case of artificial data containing two classes: 1000 "+" and 1100 "o". The solid line indicates the decision boundary training using all samples; the dash line indicates the decision boundary training using 1000 "+" and 1000 "o" samples.

It is unstable when a random initial set is used to train the SVM incrementally because the initial samples can be very unbalanced. Instead, we design a method to consider the distribution of training samples. The k-means clustering algorithm partitions the N data into k subsets S_j by minimizing the sum-of-squares criterion $J = \sum_{j=1}^{k} \sum_{\mathbf{x}_n \in S_j} |\mathbf{x}_n - \boldsymbol{\mu}_j|^2$, where $\mathbf{x}_n$ is the nth data point that belongs to S_j and $\boldsymbol{\mu}_j$ is the geometric centroid of the data points in S_j. Therefore, by selecting the initial training samples from each S_j, we can solve the unbalanced initial training samples problem.

We randomly divide the positive and negative training samples into respective K groups. The reason is when there is a large number of training samples, it is time-consuming to apply the k-means algorithm. We separate each group into k sub-clusters using the k-means clustering algorithm. Then, we choose the sample that is the closest to the cluster mean. There are $1/k$ of the total training samples to be chosen as the initial training samples. Variables K and k are determined by the size of total training samples. In Fig. 1, we use $K = 10$ and $k = 10$. Therefore, we can avoid generating the poor dash-line decision boundary and obtain the correct solid-line boundary.

2.3. Active query of new training samples

After training, the decision function of the SVM can be written as

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x}) + b = \sum_{i=1}^{l} \alpha_i y_i K(\mathbf{x}, \mathbf{x}_i) + b, \tag{12}$$

where $\mathbf{w}$ is the constructed SV in feature space to be determined by

$$\mathbf{w} = \sum_{i=1}^{s} \alpha_i y_i \mathbf{\Phi}(\mathbf{x}_i) = (w_1, w_2, \ldots, w_d), \tag{13}$$

where s is the number of SVs and d is the dimension in feature space.

An upper bound for the expected error of the SVM is given by

$$E[e] \leqslant \frac{E[D^2/M^2]}{l}, \tag{14}$$

where D is the diameter of the minimum sphere enclosing all the training samples in the feature space, and M is the margin of SVM. If the training samples are separated without errors by the optimal hyperplane, another upper bound for the expected value of errors for the SVM on the test samples can be obtained by the ratio between the expected value of the number of SVs and the number of training samples as

$$E[e] \leqslant \frac{E[s]}{l}, \tag{15}$$

where s is the number of SVs.

There are three methods to estimating the test errors by Eqs. ((3), (14), and (15)). If we use $\eta = 0.01$ in Eq. (3), the probability is 99%. Since the VC dimension of the set of oriented hyperplanes in $\mathbf{R}^n$ is $n + 1$ [3], the VC dimension in feature space $\mathbf{R}^d$ is $h = d + 1$. If the dimension d can be obtained explicitly, we use Eq. (3) to calculate the upper bound of the actual error. For a linear SVM, it is obvious that the VC dimension is $h = n + 1$. For the second-degree polynomial SVM with kernel

$$K(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x} \cdot \mathbf{y})^2, \tag{16}$$

the corresponding vector in feature space is

$$\mathbf{\Phi}(\mathbf{x}) = (\sqrt{2}x_1, \ldots, \sqrt{2}x_n, x_1^2, \ldots, x_n^2, \sqrt{2}x_1x_2, \ldots, \sqrt{2}x_{n-1}x_n), \tag{17}$$

with $d = n(n + 3)/2$. Therefore, the VC dimension for the second-degree polynomial SVM is $h = n(n + 3)/2 + 1$. For the Gaussian RBF SVM with kernel

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{(\mathbf{x} - \mathbf{y})^2}{\sigma^2}\right), \tag{18}$$

we cannot obtain h since the pattern in the input space cannot be projected to the feature space explicitly due to the infinite VC dimension.

To compute the diameter D in Eq. (14), we minimize $(D/2)^2$ subject to

$$|\mathbf{\Phi}(\mathbf{x}_i) - \mathbf{O}|^2 \leqslant (D/2)^2 \quad \forall i, \tag{19}$$

where $\mathbf{O}$ is the center of the sphere. This is a convex quadratic programming problem. If the dataset is large, it requires huge memory space and significant computation time. To avoid this, we develop the following strategy. Each attribute of training data is normalized in the range between 0 and 1. For the linear SVM, if each sample has n normalized attributes, all the training data are within an n-dimensional cube of length 1. The diameter D can be approximated as $\sqrt{n}$. For the second-degree polynomial SVM with the input n-dimensional space and the kernel given in Eq. (16), the dimension in feature space is $d = n(n + 3)/2$ as in Eq. (17). The input space is normalized, and all the training samples in the feature space are within a d-dimensional cube of length $\sqrt{2}$. The margin M in Eq. (14) can be computed as

$$M = \frac{2}{\|\mathbf{w}\|}, \tag{20}$$

where $\|\mathbf{w}\|$ is the norm of the SV $\mathbf{w}$. For the linear and second-degree polynomial SVMs, the SV $\mathbf{w}$ can be constructed by Eq. (13). For the Gaussian RBF SVM, since the projected vector in feature space cannot be obtained explicitly, we cannot calculate D and M.

The calculation of Eq. (15) is simple since it is not related to the dimension of feature space. In the training process, our goal is to select a subset from the entire training set to achieve the same performance. In order to estimate the actual error of the SVM in each incremental step using the subset, we adjust Eq. (14) to be

$$e \leqslant e_t + \frac{D^2/M^2}{l}, \tag{21}$$

and adjust Eq. (15) to be

$$e \leqslant e_t + \frac{s}{l}, \tag{22}$$

where l is the total number of training samples and s is the number of SVs for the current training set, and e_t is the error calculated on the entire training set. The term e_t is added in Eqs. (21) and (22) because only a subset is used at each incremental training step. To estimate the upper bound, the testing error of the entire training set is added. We take the minimum of Eqs. (3), (21) and (22) as the actual error

$$e = \min\left(e_t + \sqrt{\left(\frac{h(\log(2l/h) + 1) - \log(\eta/4)}{l}\right)}, e_t + \frac{(D^2/M^2)}{l}, e_t + \frac{s}{l}, 0.5\right). \tag{23}$$

We add 0.5 by assuming that the worst case performs even better than the random guess. The minimum value is taken as the actual error because all these four terms are upper bounds. We define another term e_{opt} as the error for the optimal hyperplane using the entire training dataset with respect to the estimation from the current SVM using only a subset. Since the hyperplane is obtained by minimizing the training error and maximizing the margin, we obtain e_{opt} by

$$e_{opt} = \frac{m}{l}, \tag{24}$$

where m is the number of samples tested as errors from the current training sample set. Note that if the current SVM can classify the current training samples without an error, then $e_{opt} = 0$.

We define the confidence factor C to control the selection of new training samples as

$$C = \frac{1 - e}{1 - e_{opt}}. \tag{25}$$

We define the ratio α as

$$\alpha = \tfrac{1}{2} \ln[C/(1 - C)]. \tag{26}$$

Note that 0.5 is placed in Eq. (23) to ensure the confidence factor $C \geqslant 0.5$, so $\alpha \geqslant 0$.

Each unused sample receives a weight that determines its probability for the next training step. The weight is calculated as

$$W(\mathbf{x}_i) = \begin{cases} e^{-|\alpha f(\mathbf{x}_i)|} & \text{if } \mathbf{x}_i \text{ is correctly classified,} \\ e^{(1-\alpha)|f(\mathbf{x}_i)|} & \text{otherwise,} \end{cases} \tag{27}$$

where $f(\mathbf{x}_i)$ is the decision value of pattern $\mathbf{x}_i$ using the SVM in Eq. (12). The selection of a pattern for training is controlled by α and the decision value in Eq. (27). We know that when C increases, α also increases. A high C value indicates a high confidence for the current classification. If a pattern is classified correctly, its weight is low when both the decision value and α are high. Otherwise, when $\alpha < 1$, a high value will be assigned to indicate that the wrongly classified patterns are more important, and when $\alpha \geqslant 1$, the wrongly classified patterns near the OSH are more important. New training samples will be selected according to their weights.

In the AdaBoost learning algorithm [13], a similar strategy is used to assign weights. The difference is that AdaBoost uses the final classification decision based on the combined outputs of a chain of different classifiers, while our method simply uses the final classifier that is referred to the SVM being trained using the selected final subset of training samples. Our method also differs from the near-boundary method, which selects the samples that are closest to the current separating hyperplane. From Eq. (27) we can see that when $\alpha < 1$, the wrongly classified patterns are selected with the higher $|f(\mathbf{x})|$ and the higher weight. When $\alpha \geqslant 1$, the wrongly classified patterns near the OSH are selected and the correctly classified patterns near the OSH are adjusted by decision values. The training process will repeat until no informative training sample is available. We set a threshold for the weight to be e^{-2} according to experimental results by considering both the confidence factor and the distance between the sample and the current separating hyperplane.

2.4. Elimination of non-informative training samples

In active learning, if we keep only the SVs of previous samples for the next step, it may cause bad performance; on the other hand, if we use all the previous samples, it may lead to a large number of active training samples. An et al. [10] discarded the non-informative samples by considering both the distribution of samples and the precision of the classifier. By considering the distance of a training sample to the current hyperplane and its weight, we develop the following method to actively eliminating some training samples that are not informative.

Firstly, we set a threshold on the decision value. If a sample has $|f(\mathbf{x})| < 1.1$, we keep it because it is near the hyperplane. Secondly, according to Eq. (27), we assign a weight to each sample in the current training set. We set a threshold $t = e^{-2}$. If the weight of a training sample is higher than t, we keep it for the next step. In some cases when C is high, we may have a high α. If we only use the second condition, $t = e^{-2}$, as the threshold, all the current training samples may be eliminated.

The pseudo-code of the proposed incremental training algorithm for SVMs is described as follows:

Apply the k-means clustering algorithm to select the initial training samples $Q(0)$.
Train the initial $SVM(0)$.
Assign a weight w to all the training samples using Eq. (27).
$t = 0$;
While the unused informative training sample exists $(w > e^{-2})$
  $t = t + 1$;
  Eliminate non-informative training samples v $(|f(v)| >= 1.1 \&\& w < e^{-2})$ from $Q(t)$: $Q(t) = Q(t) - v$.
  Query q training samples from unused training samples with the highest w values.
  Set $Q(t) = Q(t) \cup q$.
  Train $6SVM(t)$ using $Q(t)$.
  According to $SVM(t)$, assign w to all the training samples using Eq. (27).
End While

3. Experimental results

We use our improved incremental training algorithm with three kernels of SVMs: linear, second-degree polynomial, and Gaussian RBF. All the training samples are normalized between 0 and 1. We set the regularization constant $C = 1$ and the variance of the Gaussian RBF kernel $\sigma^2 = 1$. For comparison purposes, we use the same set of parameters for all methods.

The test sets are listed in Table 1. The face dataset contains 2429 faces and 15,228 non-faces. The 2429 face

Table 1
The test datasets

Dataset	No. of attributes	No. of class 1	No. of class 2	Total number
Face	60	2429	15,228	17,657
Cancer	9	444	239	683
Diabetes	8	500	268	768
Ionosphere	34	225	126	351
Heart	13	150	120	270
German	24	700	300	1000

Table 2
Classification rates of different methods using linear SVM

Dataset	Method					
	BatchSVM	IncrSVM random	QuerySVM	SubsetSVM	QueryStat	Our method
Face	0.9618	0.9392	0.9594	0.9447	0.9096	0.9616
Cancer	0.9716	0.9701	0.9716	0.9687	0.9716	0.9731
Diabetes	0.7684	0.7524	0.7605	0.7711	0.7763	0.7711
Ionosphere	0.8824	0.8559	0.8706	0.85	0.8676	0.8824
Heart	0.8407	0.8222	0.8444	0.8407	0.8407	0.8407
German	0.7660	0.7440	0.7510	0.7510	0.7060	0.7720
Average Difference	0	−0.0178	−0.0056	−0.0108	−0.0198	0.0017

Table 3
Classification rates of different methods using second-degree polynomial SVM

Dataset	Method					
	BatchSVM	IncrSVM random	QuerySVM	SubsetSVM	QueryStat	Our method
Face	0.9858	0.9077	0.9815	0.9689	0.9605	0.9863
Cancer	0.9716	0.9701	0.9701	0.9716	0.9642	0.9716
Diabetes	0.7789	0.7611	0.7671	0.7724	0.7303	0.7763
Ionosphere	0.9176	0.8647	0.8618	0.8676	0.8824	0.9176
Heart	0.8148	0.8037	0.8037	0.7741	0.7852	0.8259
German	0.7460	0.7030	0.7280	0.7030	0.6850	0.7310
Average Difference	0	−0.0341	−0.0171	−0.0262	−0.0345	−0.0010

images are obtained from *the Center for Biological and Computational Learning* (CBCL) at *Massachusetts Institute of Technology* (MIT). The faces images can be retrieved from URL: http://cbcl.mit.edu/software-datasets/FaceData2.html. We randomly collected 15,228 non- face training samples. All the samples are normalized to be 19 × 19 pixels. Each sample is represented by the top 60 PCA values. The other datasets of cancer, diabetes, ionosphere, heart, and German are obtained from the Repository of Machine Learning Databases of *the University of California*, *Irvine* (UCI) [14].

We compare our method with five existing methods. The first method, called BatchSVM [1], uses all the positive and negative training samples to train the SVM. Note that this method requires a large amount of training samples. The second method, called IncrSVMRandom, selects new training samples randomly at each incremental step. The third method, called QuerySVM [7,9,10], is to query and collect new training samples near the current separating hyperplane. The fourth method, called SubsetSVM [8], is an incremental learning procedure by partitioning the training dataset into subsets and adding only SVs in each incremental step. The fifth method, called QueryStat [12], is a probabilistic active learning procedure using a distribution determined by the current separating hyperplane and a confidence factor.

Each dataset contains two classes, and each attribute is normalized in the range between 0 and 1. We perform experiments 10 times, with each time taking different 90% samples from class 1 and class 2 as the training samples and the rest of 10% samples as the test samples. The overall classification rate is the average of the 10 classification rates. For linear and second-degree polynomial SVMs, we calculate the VC dimension h, the diameter D, and the margin M. Therefore, we can use Eq. (17) to calculate the error. The comparisons of different methods of using the linear SVM are provided in Table 2. The results of using the second-degree polynomial SVM are given in Table 3. For the Gaussian RBF SVM, we use Eq. (16) to estimate the actual error, and the results are shown in Table 4.

We use the BatchSVM method as the basis to be compared with other methods, and calculate the average difference of classification rates as shown in the bottom row of Tables 2–4. We observe that the overall performance of our method outperforms other methods. In most cases, the IncrSVMRandom method performs worse than the QuerySVM method. For linear and Gaussian RBF SVMs, our method outperforms the BatchSVM method slightly. The QueryStat method performs badly in our experiments because it only keeps the SVs of the previous training samples.

To justify our strategy of eliminating non-informative training samples, we conduct experiments on the face dataset that contains 2429 face and 15,228 non-face samples. The linear SVM is used for training and testing. The initial training samples are selected using the strategy in Section 2.2, and the new samples are queried using the strategy in Section 2.3. In each incremental training step, 200 new samples are selected. Table 5 shows the number of training samples and classification rate in each step for one round of experiment. We listed this result because it is

Table 4
Classification rates of different methods Using Gaussian RBF SVM

Dataset	Method					
	BatchSVM	IncrSVM random	QuerySVM	SubsetSVM	QueryStat	Our method
Face	0.9864	0.9533	0.9854	0.9789	0.9334	0.9863
Cancer	0.9716	0.9701	0.9731	0.9731	0.9687	0.9731
Diabetes	0.7750	0.7684	0.7789	0.7645	0.7763	0.7750
Ionosphere	0.9380	0.9312	0.9412	0.9324	0.9265	0.9382
Heart	0.8185	0.8037	0.8148	0.8000	0.8111	0.8185
German	0.7340	0.7290	0.7310	0.7380	0.7290	0.7340
Average Difference	0	−0.0113	0.0002	−0.0061	−0.0131	0.0003

Table 5
The number of training samples and classification rate in each incremental training step for one round of experiment

Step	No. of keeping all the samples	No. of newly queried samples	No. of eliminating non-informative samples	Classification rate
1	1579		1579	0.9660
2	1779	200	482	0.9592
3	1979	200	640	0.9711
4	2179	200	801	0.9683
5	2379	200	985	0.9700
6	2579	200	1176	0.9660
7	2779	200	1372	0.9745
8	2979	200	1564	0.9734
9	3179	200	1764	0.9745
10	3379	200	1964	0.9745
11	3579	200	2164	0.9734
12	3739	50	2364	0.9745
13	3789		2414	0.9745

close to the average performance. In one round of experiment, 1764 samples (242 faces+1522 non-faces) are set as testing samples and 15,893 samples (2187 faces+13,706 non-faces) are set as training samples.

From Table 5, totally 3789 samples are involved in training. It is about 23.8% of the total training samples. Using the non-informative training samples eliminating method, 2414 training samples are kept in the final training set. It is about 63.81% (2414/3789) of the totally queried samples. For the classification rate, the initial training rate is 0.9660. The final classification rate is slightly higher than the initial training rate. This reflects the effectiveness of both the initial training sample selection method and the incremental training method. Note that the classification rate does not always increase in each step. In our algorithm, the process continues until all the informative samples are used. It would be an interesting research issue to investigate theoretically when to stop the training process while achieving optimal or anticipated performance.

4. Conclusions

In this paper, we have presented an improved incremental training algorithm for SVMs. The k-means clustering algorithm is used to collect the initial training samples. In active querying, a weight is assigned to each sample according to its distance to the current separating hyperplane and a confidence factor indicating the degree that the current separating hyperplane is the optimal solution. The confidence factor is obtained from the estimated generalization error of the current SVM which is calculated from the upper error bounds of the SVM. Non-informative samples are eliminated from the active training set by considering their distances to the separating hyperplane and their weights. We compare the performance of our method with other methods using the linear, the second-degree polynomial, and the RBF SVMs. Experimental results show the robustness of the proposed method.

References

[1] C. Cortes, V. Vapnik, Support-vector networks, Mach. Learn. 20 (1995) 273–297.
[2] V. Vapnik, The Nature of Statistical Learning Theory, Springer, New York, 1995.
[3] C.J.C. Burges, A tutorial on support vector machines for pattern recognition, IEEE Trans. Data Min. Knowl. Discovery 2 (2) (1998) 121–167.
[4] V. Vapnik, Statistical Learning Theory, Wiley, New York, 1998.
[5] B. Heisele, T. Serre, S. Prentice, T. Poggio, Hierarchical classification and feature reduction for fast face detection with support vector machines, Pattern Recognition 36 (9) (2003) 2007–2017.
[6] F.Y. Shih, S. Cheng, Improved feature reduction in input and feature spaces, Pattern Recognition 38 (5) (2005) 651–659.
[7] G. Schohn, D. Cohn, Less is more: active learning with support vector machines, Proceedings of the 17th International Conference on Machine Learning, Stanford University, CA, June 2000, pp. 839–846.
[8] N.A. Syed, H. Liu, K.K. Sung, Incremental learning with support vector machines, Proceedings of the International Joint Conference on Artificial Intelligence, Stockholm, Sweden, July 1999.
[9] C. Campbell, N. Cristianini, A. Smola, Query learning with large margin classifiers, Proceedings of the 17th International Conference on Machine Learning, Stanford University, CA, June 2000, pp. 111–118.
[10] J.-L. An, Z.-O. Wang, Z.-P. Ma, An incremental learning algorithm for support vector machine, Proceedings of the Second International Conference on Machine Learning and Cybernetics, Xi'an, China, November 2003, pp. 1153–1156.
[11] H.T. Nguyen, A. Smeulders, Active learning using pre-clustering, Proceedings of the 21st International Conference on Machine Learning, Alberta, Canada, July 2004.

[12] P. Mitra, C.A. Murthy, S.K. Pal, A probabilistic active support vector learning algorithm, IEEE Trans. Pattern Anal. Mach. Intell. 26 (3) (2004) 413–418.

[13] R. Duda, P. Hart, D. Stork, Pattern Classification, Wiley-Interscience Publication, New York, 2001.

[14] C.L. Blake, C.J. Merz, UCI Repository of Machine Learning Databases, ⟨http://www.ics.uci.edu/∼mlearn/MLRepository.html⟩, Department of Information and Computer Science, University of California, Irvine, CA, 1998.

About the Author—SHOUXIAN CHENG received the B.S. degree in Material Science from Beijing University of Chemical Technology, Beijing, China, in 1995, and the M.S. and Ph.D. degrees in Computer Science from New Jersey Institute of Technology, respectively, in 2001 and 2005. His research interests include image processing, pattern recognition and machine learning.

About the Author—FRANK Y. SHIH received the B.S. degree from National Cheng-Kung University, Taiwan, in 1980, the M.S. degree from the State University of New York at Stony Brook, in 1984, and the Ph.D. degree from Purdue University, West Lafayette, Indiana, in 1987, all in Electrical Engineering. He is presently a professor jointly appointed in the Department of Computer Science (CS), the Department of Electrical and Computer Engineering, and the Department of Biomedical Engineering at New Jersey Institute of Technology, Newark, NJ. He currently serves as the Director of Computer Vision Laboratory.

Dr. Shih is currently on the Editorial Board of the International Journal of Pattern Recognition, the International Journal of Pattern Recognition and Artificial Intelligence, the International Journal of Internet Protocol Technology, and the Journal of Internet Technology. He previously served as an Associate Editor of the International Journal of Systems Integration and the International Journal of Information Sciences. He has contributed as a steering member, committee member and session chair for numerous professional conferences and workshops. He was the recipient of the Research Initiation Award from the National Science Foundation in 1991. He won the Honorable Mention Award from the International Pattern Recognition Society for Outstanding Paper and also won the Best Paper Award in the International Symposium on Multimedia Information Processing. He has received several awards for distinguished research at New Jersey Institute of Technology. He has served several times on the Proposal Review Panel of the National Science Foundation.

Dr. Shih holds the research fellow for the American Biographical Institute and the IEEE senior membership. He has published seven book chapters and over 180 technical papers, including 80 in well-known prestigious journals. His current research interests include image processing, computer vision, sensor networks, pattern recognition, bioinformatics, information security, robotics, fuzzy logic and neural networks.